

SECURITY AUDIT

Symbiotic

Async/Account , Scope 3

FINAL



REPORT



Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Symbiotic – Async/Account – Audit Report – Scope 3
Website	symbiotic.fi
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/symbioticfi/core-mirror/blob/42b4c2e5dc36ded4b0390534926a87e48ce40bfe/src/contracts/adapters/Il-adapter/common/AsyncRedeemAccount.sol
Resolution 1	https://github.com/symbioticfi/core-mirror/tree/67bad5bc94cf05f5dc8139b3f1a45669a778dea1

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	1	1				
Low	6	1		5		
Informational	7	1	1	5		
Governance						
Total	14	3	1	10		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

AsyncRedeemAccount

The `AsyncRedeemAccount` contract is an abstract base account for integrating with ERC-7540 async redeem vaults, extending `CoolDownAccount` with request-based redemption handling. It tracks the identifiers of its open redemption requests in the `requestIds` array and resolves the target async redeem vault dynamically by querying the ERC-7575 share token for the vault associated with the account asset. `_totalAssets` values the account as the sum of all pending redeem requests converted to assets at current prices plus the claimable amount reported by `maxWithdraw` at fulfillment prices.

Invariants

- Every id in `requestIds` has a nonzero `pendingRedeemRequest` in the vault after `_finalizeRequests` runs
- `requestIds` contains no duplicate request ids
- After `_requestRedeem`, the account's entire token-to-redeem balance has been submitted to the vault for redemption
- `_totalAssets` equals the asset value of all pending requests plus the currently claimable withdraw amount

Issue_01	<code>_finalizeRequests</code> can pop a fulfilled redemption without claiming it, stranding assets from the automated flow
Severity	Medium
Description	In <code>_finalizeRequests</code>, the <code>maxRedeem</code> function is called, and if <code>>0</code>, <code>redeem</code> is called to claim the redeemed assets. <pre>uint256 maxRedeem = IAsyncRedeemVault(asyncRedeemVault).maxRedeem(address(this)); if (maxRedeem > 0) { IAsyncRedeemVault(asyncRedeemVault).redeem(maxRedeem, address(this), address(this)); }</pre> After this, the redemption request is popped if <code>pending</code> is 0. The issue is that under certain conditions, <code>maxRedeem</code> can return 0. This happens when the admin of Centrifuge pauses token transfers via the hooks. <code>maxRedeem</code> calls <code>_canTransfer</code>, which can be used to disable this function. <pre>if (!_canTransfer(vault_, user, address(0), shares)) return 0;</pre> In that case, the request will be popped without redeeming the funds, since <code>maxRedeem = 0</code>. Future <code>_finalizeRequests</code> calls will also fail to redeem funds due to the early exit in case of an empty requests array. <pre>uint256 length = requestIds.length; if (length == 0) { return; }</pre> So if the Centrifuge vault is ever paused and then <code>sync</code> is called in that state, requests that have been redeemed but not claimed can get popped and then be unable to be claimed. This will persist until there are fresh funds that can be redeemed in the account, and the request ID gets added again in <code>_requestRedeem</code>, resulting in an availability DOS.

Recommendations	Consider allowing redemptions to process even when requestIds are empty, since in Centrifuge, all requests are fungible. Otherwise, add an admin-gated redeem function to redeem positions in case requestId is empty but positions exist.
Comments / Resolution	Fixed by following recommendations. <code>_finalizeRequests</code> will attempt to redeem even if there are no <code>requestIDs</code> .

Issue_02	Claimable redemption assets can disappear from <code>totalAssets</code> during transfer restrictions
Severity	Low
Description	<code>AsyncRedeemAccount._totalAssets</code> calculates the value of fulfilled and claimable redemptions using <code>maxWithdraw</code>. <pre>assets += AsyncRedeemVault(asyncRedeemVault).maxWithdraw(address(this));</pre> However, Centrifuge's <code>maxWithdraw</code> is not only an accounting function. It also checks whether the assets can currently be transferred. If Centrifuge pauses or restricts transfers, <code>maxWithdraw()</code> can return zero: <pre>if (!_canTransfer(vault_, user, address(0), shares)) return 0;</pre> The fulfilled redemption assets still belong to the account, but <code>AsyncRedeemAccount</code> temporarily reports them as having zero value. This creates an artificial decrease in <code>totalAssets</code> and the corresponding vault share price. Users may deposit while the claimable assets are excluded and receive more shares than they should. When transfers are enabled again, <code>maxWithdraw</code> includes the assets and the value suddenly returns. The newly minted shares then capture part of that recovered value. The artificial recovery may also cause performance fees to be charged again on value that was never actually lost.
Recommendations	Value claimable assets separately from <code>maxWithdraw</code> , so transfer restrictions do not remove them from <code>totalAssets</code> .
Comments / Resolution	Acknowledged.

Issue_03	Pricing inconsistency in <code>_totalAssets</code>
Severity	Low
Description	In <code>totalAssets</code>, the parent Account contract values the vault tokens using <code>IOracle[ORACLE].getPrice()</code>. <pre>function _tokenToRedeemToAssets(uint256 amount, uint256 rate) internal view returns (uint256) { return amount > 0 ? amount.mulDiv(rate * _unit, TO_ASSETS_DIVISOR) : 0; }</pre> The pending requests are, however, valued using <code>convertToAssets</code> directly. <pre>assets += IAsyncRedeemVault(asyncRedeemVault) .convertToAssets(IAsyncRedeemVault(asyncRedeemVault).pendingRed eemRequest(requestIds[i], address(this)));</pre> This creates an inconsistency, since the oracle price implements a min-max price gate, which is skipped when using raw <code>convertToAssets</code>.
Recommendations	Consider using the oracle price for both components.
Comments / Resolution	Fixed.

Issue_04	The account cannot deal with cancelled redemptions
Severity	Low
Description	Centrifuge vault expects users to call <code>claimCancelRedeemRequest</code> if part of their redemption is cancelled. The account, however, does not implement any way to do so.
Recommendations	Consider adding an admin-gated function to call <code>claimCancelRedeemRequest</code> .
Comments / Resolution	Acknowledged.

Issue_05	Centrifuge redeem can leave dust amounts of funds unprocessed
Severity	Low
Description	The Centrifuge contract natspec here warns that when using <code>redeem</code>, <i>When claiming redemption requests using <code>redeem()</code>, there can be some precision loss leading to dust. It is recommended to use <code>withdraw()</code> to claim redemption requests instead.</i> Thus, using <code>redeem</code> can lead to dust funds being left in the vault.
Recommendations	Consider following the recommendation and using <code>withdraw</code> instead.
Comments / Resolution	Acknowledged.

Issue_06	Fulfillment can cause sudden NAV changes
Severity	Low
Description	Pending requests are valued using the current <code>convertToAssets</code> price. After fulfillment, <code>maxWithdraw</code> uses the actual execution value. If these values differ, <code>totalAssets</code> changes suddenly. Users may deposit before an increase or withdraw before a decrease.
Recommendations	Use consistent pricing before and after fulfillment.
Comments / Resolution	Acknowledged.

Issue_07	Instant redemption swap reverts when Centrifuge <code>requestRedeem</code> fails
Severity	Low
Description	<code>LiquidLaneAdapter._swap</code> moves shares into the redeem account and then calls sync in the same transaction. Sync first finalizes claimable redemptions, then, when cooldown allows and the account holds shares, submits the full share balance via Centrifuge <code>requestRedeem</code>. There is no isolation: if that call reverts, the entire swap reverts and the market maker cannot complete instant redemption. That couples swap liveness to Centrifuge request-side conditions unrelated to the swap itself. In the referenced path, <code>requestRedeem</code> can fail on share transfer restrictions, a pending cancel, messaging/subsidy failures, or similar vault-side checks. The account owner is the curator, not the adapter, so swaps only trigger a new request on the first sync or after cooldown, but once that window is open, a failing request still aborts the swap. Idle shares are already accepted during cooldown and priced by the oracle, so failing the market-maker flow solely because a later request cannot be submitted is unnecessary coupling and an avoidable denial of the instant-redeem path when the pool or Centrifuge stack refuses or cannot process requests.
Recommendations	Isolate request submission from the swap path: catch or skip a failing <code>requestRedeem</code> (and optionally separate finalize from request) so sync can leave shares idle when Centrifuge will not accept a new request, without reverting <code>_swap</code> .
Comments / Resolution	Acknowledged.

Issue_08	getPrice is called even for a 0 amount
Severity	Informational
Description	In <code>_totalAssets</code>, the tokens are valued using the following snippet. <pre><i>assets = _tokenToRedeemToAssets(amount, IOracle(ORACLE).getPrice());</i></pre> The issue is that <code>getPrice</code> is called even when it is unnecessary, if <code>amount = 0</code>. This is avoided in the Account contract, since <code>getPrice</code> is called only if necessary (<code>amount > 0</code>). <pre><i>function _tokenToRedeemToAssets(uint256 amount) internal view virtual returns (uint256) { return amount > 0 ? _tokenToRedeemToAssets(amount, IOracle(ORACLE).getPrice()) : 0; }</i></pre> This means that if the oracle is returning a faulty value, the <code>_totalAssets</code> function can revert even if it does not hold any tokens using that oracle.
Recommendations	Consider using the same <code>_tokenToRedeemToAssets(uint256 amount)</code> function as present in the Account contract.
Comments / Resolution	Fixed.

Issue_09	Cross-asset redemption proceeds are valued 1:1 after decimal normalization
Severity	Informational
Description	In AsyncRedeemAccount, when REDEMPTION_TOKEN differs from the account asset [_asset], claimable withdrawals and held redemption-token balances are converted into account-asset units only by matching decimals. No oracle or market price is used, so the contract treats one unit of the redemption token as equal to one unit of the account asset. That assumption fails whenever the tokens trade at different prices, including same-peg stablecoins during a depeg or temporary drift.
Recommendations	Price redemption-token amounts with an oracle (or other market rate) when converting to _asset, or restrict REDEMPTION_TOKEN to equal _asset and enforce that at deployment.
Comments / Resolution	Acknowledged.

Issue_10	Vault changes can make redemption requests inaccessible
Severity	Informational
Description	<code>AsyncRedeemAccount</code> does not store which vault created each request. Instead, it always reads the current vault: <pre><i>function _asyncRedeemVault() internal view virtual returns (address) { return IERC7575Share[TOKEN_TO_REDEEM].vault[_asset]; }</i></pre> If Centrifuge removes or replaces that vault while a request is active, the account stops using the original vault. This can make <code>totalAssets()</code> and <code>sync</code> revert or exclude the old redemption from accounting and finalization.
Recommendations	Store the vault used for each request until the request is settled.
Comments / Resolution	Acknowledged.

Issue_11	Deallocate sweeps realized asset balance only, not claimable redemptions
Severity	Informational
Description	<code>LiquidLaneAdapter.deallocate</code> pulls each account's ERC-20 asset balance and sends it to the adapter for the delegator. It does not call sync or redeem first. For async redeem accounts, fulfilled redemptions sit as claimable vault value (<code>maxWithdraw</code>) until sync runs <code>_finalizeRequests</code> and redeem moves assets onto the account. Until then, <code>deallocate</code> and <code>freeAssets</code> only see the realized <code>balanceOf(asset)</code>, while <code>totalAssets</code> already counts that claimable amount. Funds are not lost or stealable; they just stay claimable until a separate sync, so a deallocate that expects "all available proceeds" can return less than the accounted claimable value.
Recommendations	Document that operators should <code>sync</code> (or otherwise finalize claimable redemptions) before relying on <code>deallocate/freeAssets</code> for full proceeds, or have deallocate finalize claimable redemptions before sweeping balances.
Comments / Resolution	Acknowledged.

Issue_12	Unsupported ERC7540 operations
Severity	Informational
Description	The contract states that it is built to support ERC7540 vaults. However, certain operations in the contract are only applicable to Centrifuge vaults, and don't actually follow the ERC 7540 vault standard. 1. <code>maxWithdraw</code> in ERC7540 inherits the definition from ERC4626, representing the maximum amount that can be redeemed by the user. However, in Centrifuge and the account contract here, <code>maxWithdraw</code> is expected to represent the redeemed but not yet claimed amount. 2. <code>_asyncRedeemVault</code> defined in the contract is only applicable for ERC7575 multi-currency vaults. It is not applicable for base ERC7540 vaults.
Recommendations	Consider either documenting that this contract is for Centrifuge only, or implementing ERC7540 spec functions here and then overriding them in <code>CentrifugeAccount</code> .
Comments / Resolution	Partially fixed. <code>_asyncRedeemVault</code> now supports normal ERC7540 operations.

Issue_13	Pending redemptions accounting assumes a live request price
Severity	Informational
Description	ERC 7540 does not require a redemption request to remain priced at the current <code>convertToAssets</code> rate, <code>AsyncRedeemAccount</code> nevertheless values every pending request by passing its pending shares to the vault's current <code>convertToAssets</code> function. A compatible vault that fixes the redemption rate at submission or at another point before fulfillment can therefore be overvalued or undervalued while the request remains pending. The incorrect value would propagate into <code>VaultV2</code> NAV, share pricing, fees, and allocation limits. The current Centrifuge integration is not affected because its pending requests intentionally follow the live price, and the exact asset value is established during fulfillment.
Recommendations	Document that <code>AsyncRedeemAccount</code> only supports vaults whose pending requests can be valued using the current <code>convertToAssets</code> rate.
Comments / Resolution	Acknowledged.

AsyncRedeemOracle

The `AsyncRedeemOracle` contract prices the shares of an ERC-7540 async redeem vault in 1e18 precision, extending the bounded Oracle base, which rejects any price outside its immutable `MIN_PRICE` and `MAX_PRICE` range. It serves as the price source for accounts in the II-adaptor system that hold the vault's share token as their token to redeem.

Invariants

- `getPrice` always returns a value within [`MIN_PRICE`, `MAX_PRICE`] or reverts
- The returned price is expressed in 1e18 precision regardless of the share and asset token decimals
- `ASYNC_REDEEM_VAULT`, `SHARE_UNIT`, and `ASSET_UNIT` are fixed at deployment and never change

Issue_14	Oracle and account vaults are not validated
Severity	Informational
Description	AsyncRedeemOracle permanently prices shares through its configured <code>ASYNC_REDEEM_VAULT</code>, while AsyncRedeemAccount resolves <code>TOKEN_TO_REDEEM.vault[_asset]</code>. Initialization does not verify that these vaults match or that the oracle vault uses the account's token and asset. Incorrect configuration can therefore price shares through one vault while routing redemptions through another. This may produce incorrect swap quotes and cause <code>totalAssets</code> to change after shares enter the redemption process.
Recommendations	Validate during initialization that the oracle vault matches the account vault, asset, and share token.
Comments / Resolution	Acknowledged.